

# Knowing Spark

Up Close

# Agenda

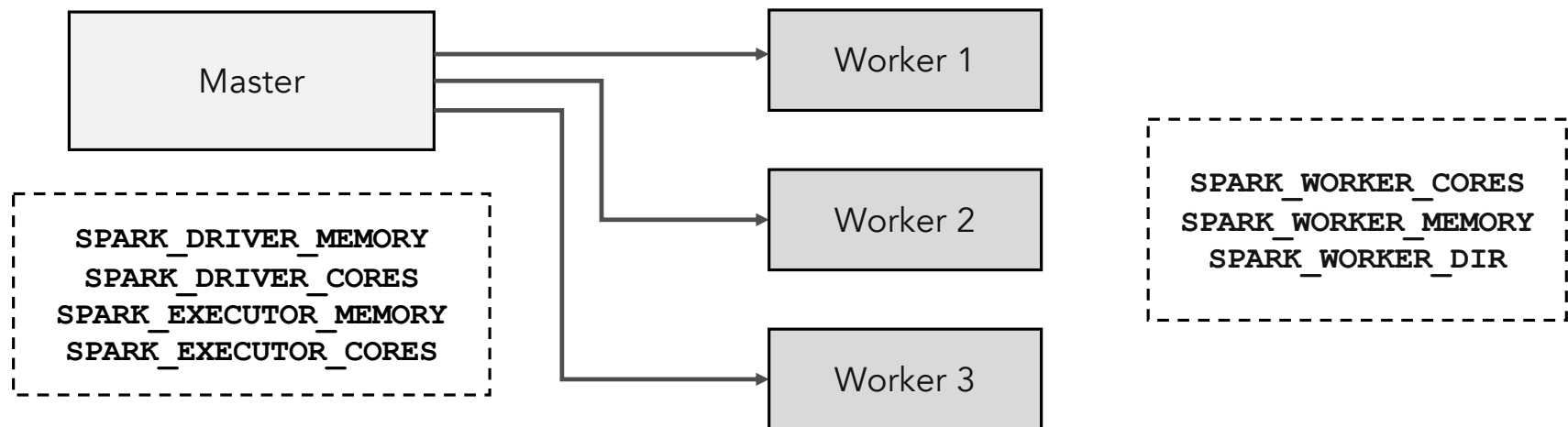
1. Driver & Executor
2. Spark Standalone Mode
3. Spark UI
4. Standalone Mode on EC2
5. DataFrames
6. DataFrame Reader & Writer
7. PySpark Module
8. SparkSession
9. File Sources
10. Input Partitions

# Standalone Deployment & UI

# Driver & Executor - RECAP

- Driver Process
  - Coordinates with Resource Manager for compute resources.
  - Analyzes the code and creates execution plan.
  - Schedules jobs and tasks.
  - Tracks the status of the jobs. Failed tasks are replaced with new tasks by Driver.
- Executor Process
  - Runs the actual transformation code based on the access plan provided by Driver.

# Standalone on Laptop

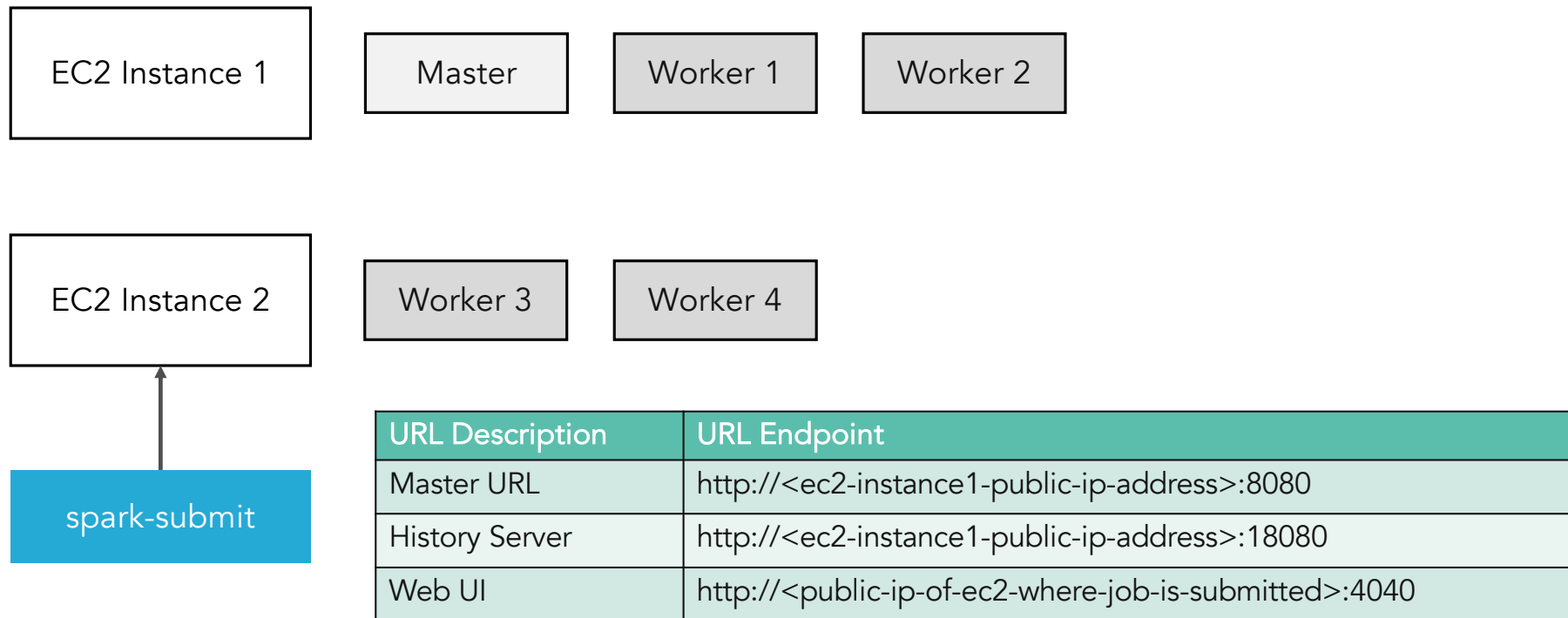


Let's create Standalone Spark cluster in our laptops

# Spark UI

- Master Node (Standalone deployment mode) – <http://localhost:8080>
  - Configuration in Spark environment file.
  - Start master
  - Start worker
- Web UI or Application UI – <http://localhost:4040>
  - Run spark application
- History UI – <http://localhost:18080>
  - Configure history logs
  - Start history server

# Standalone on 2 EC2 Instances

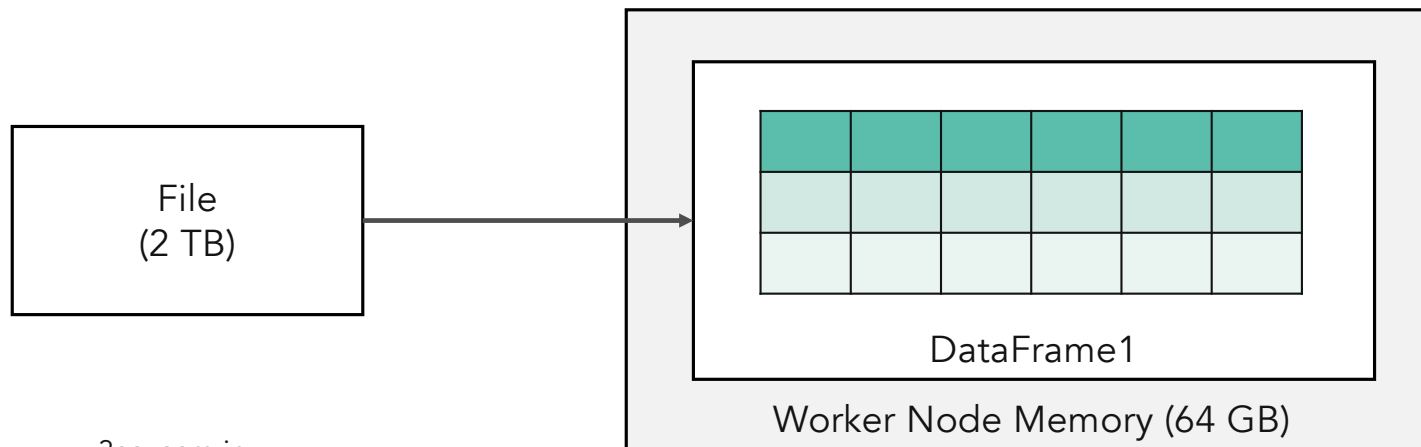


# DataFrames



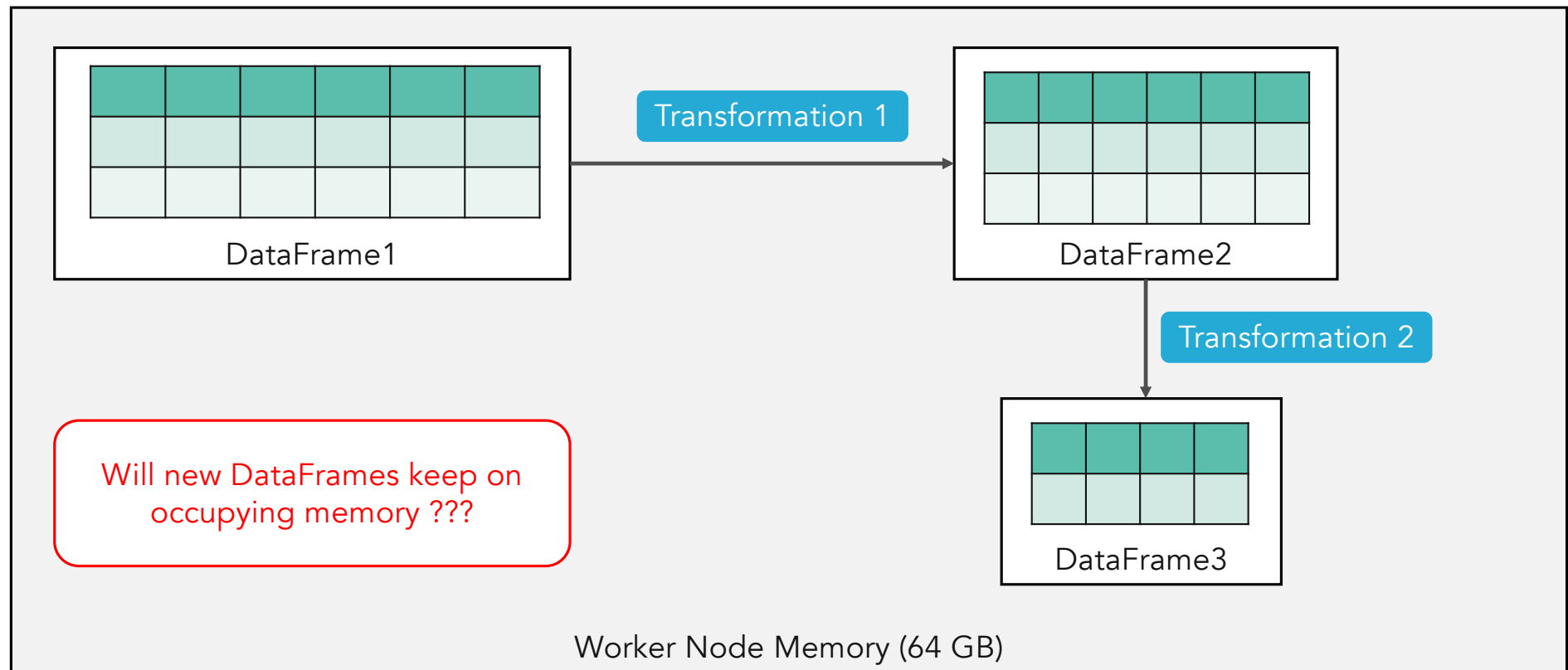
# DataFrames

- Table Recap – Contains data along with a schema.
  - Schema – Structure of the table. Name and data type of columns.
  - Data – Rows with values of each column.
- DataFrame – In-memory table created by Spark.
  - Schema – Structure, columns and data types.
  - Data – Read from files, databases etc.

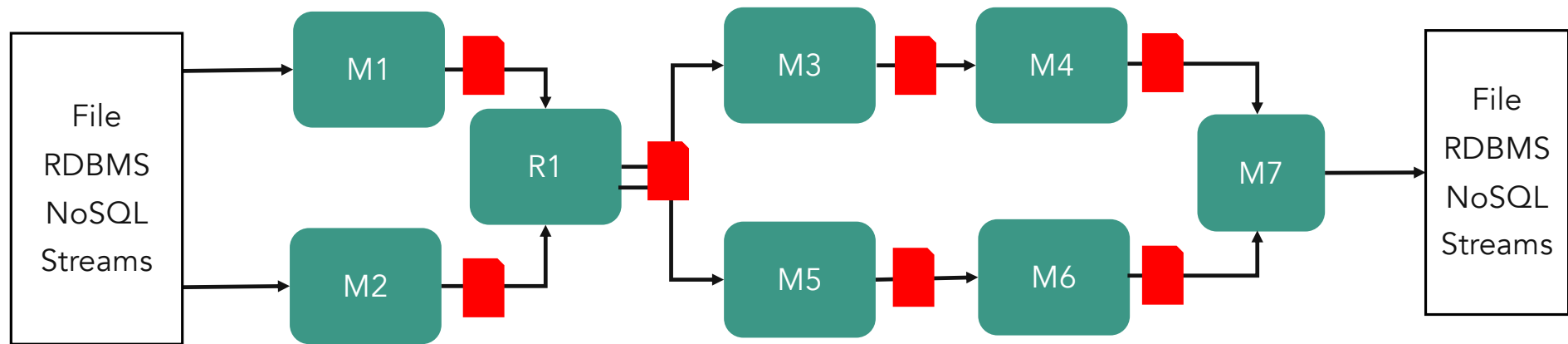


How such a big file fit in memory?

# DataFrames & DAG



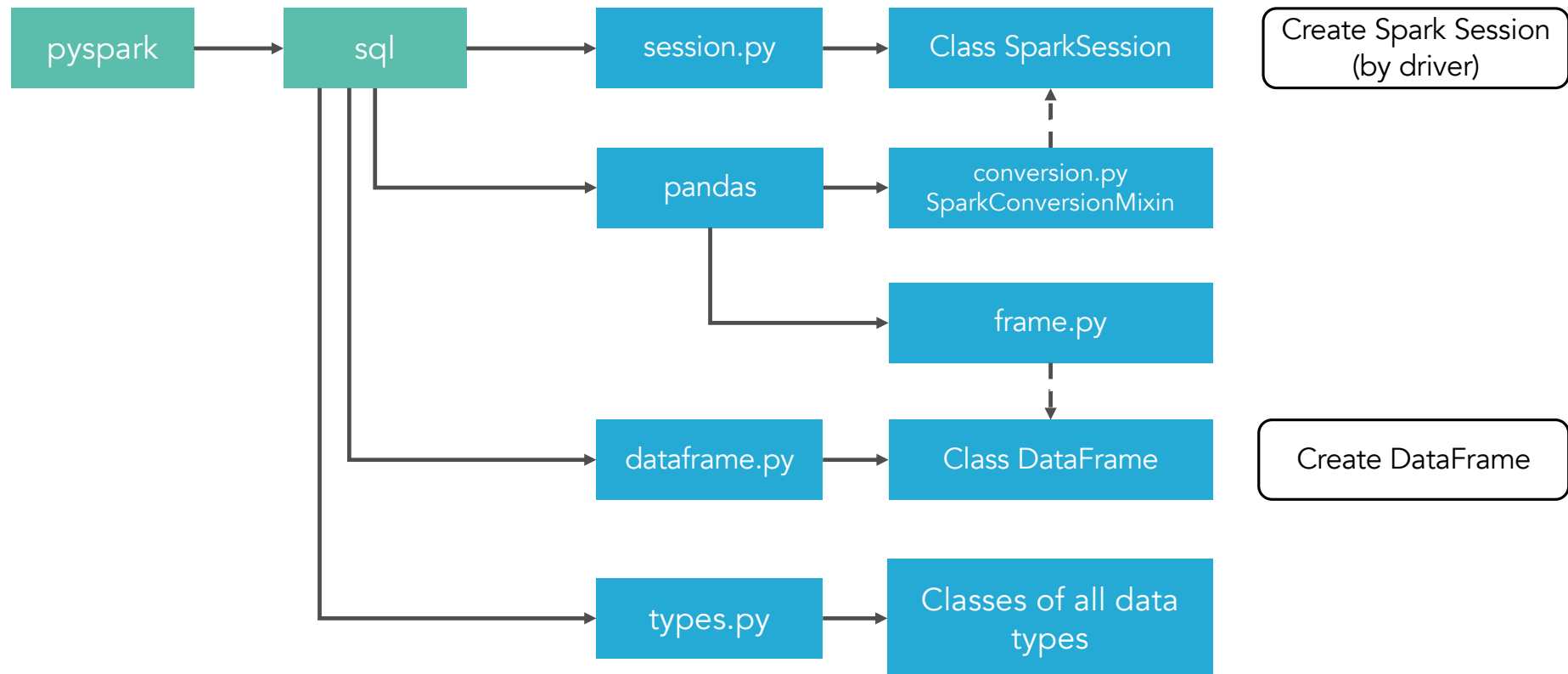
# Spark IO?



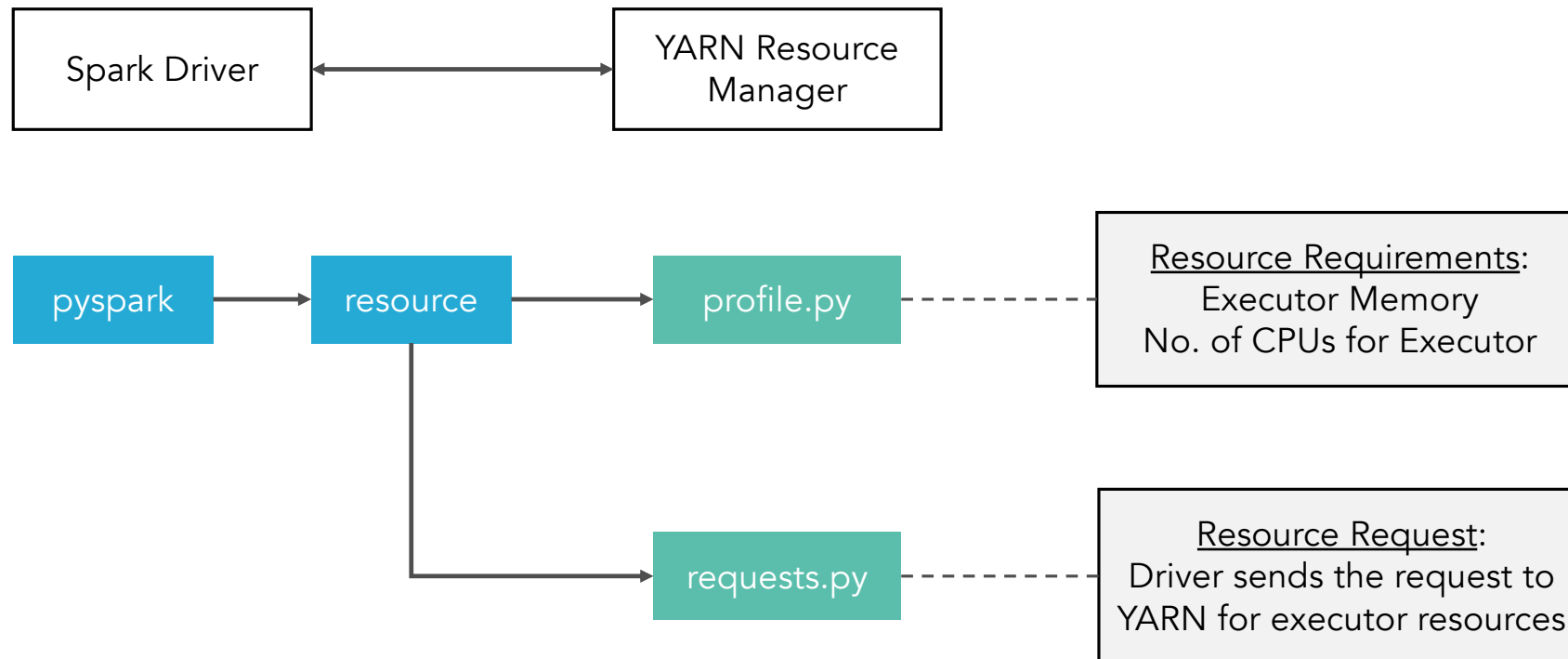
- Any source would be read into DataFrames.
- Any target would be written from DataFrames.
- DataFrame is the building block of Spark.
- Built on top of RDDs. For better performance and best practice, we should use DataFrame APIs only.

# PySpark Module

# PySpark Module



# PySpark Module



# SparkSession

- Pre-requisite: Python OOP basics.
- SparkConversionMixin – Converts from Pandas to Spark.
- conf – Runtime Spark configuration. Get and Set.
- master – Where to run Spark. YARN or local machine.
- appName – Spark application name to be shown in Spark UI.
- getOrCreate – Get an existing SparkSession or create a new one.
- create – Create a new SparkSession.
- builder – construct SparkSession.
- sql – Run SQL in Spark. Spark SQL.

Class SparkSession (inherits SparkConversionMixin)

Class Builder

def conf( )

def master( )

def appName( )

def getOrCreate( )

def create( )

def builder( )

def sql( )

def sparkContext( )

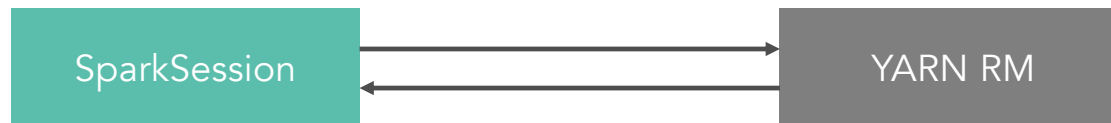
def createDataFrame( )

def readStream( )

# SparkSession

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName('SparkBatch').getOrCreate()
```

"spark" - variable, object or class ?



```
from pyspark.sql import SparkSession
from pyspark import SparkContext

spark = SparkSession(SparkContext())
spark = spark.builder
spark = spark.appName('SparkIntroduction')
spark = spark.getOrCreate()
```

Executor 1

Executor 2

Executor 3

Spark  
Transformation  
Code



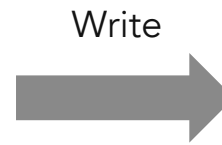
# File Sources

# File Source & Targets

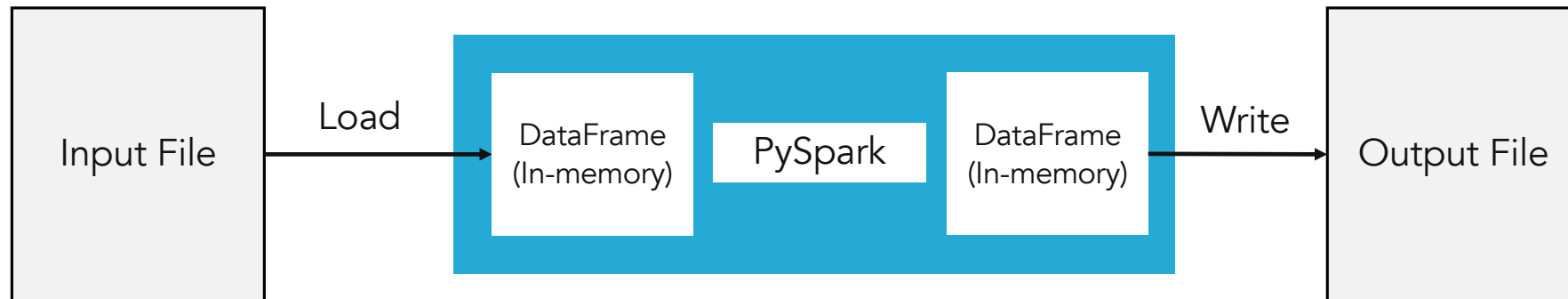
- CSV
- Parquet
- Avro
- JSON



Spark (PySpark)  
Processing



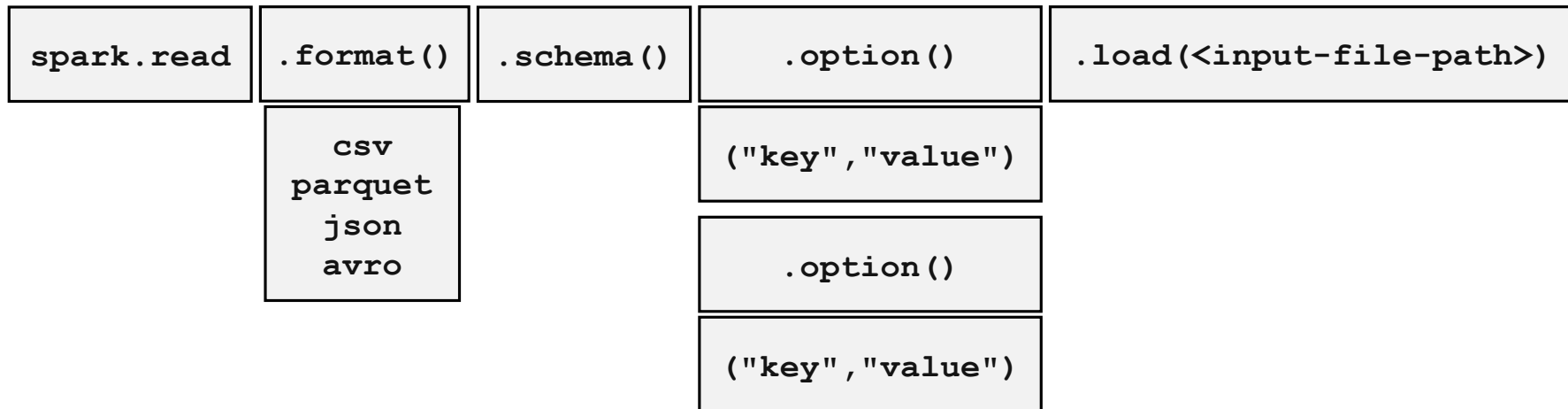
- CSV
- Parquet
- Avro
- JSON



# DataFrame Reader & Writer

- Read & Write
- DataFrame Reader

```
spark = SparkSession
```



# DataFrame Reader & Writer

- Read & Write
- DataFrame Reader
- DataFrame Writer

**df.write**

**.mode()**

**.save(<output-file-path>)**

**append**  
**overwrite**  
**error**  
**ignore**

Append = Append to existing file.

Overwrite = Overwrite existing data.

Error = Error if file and data exists.

Ignore = Don't write data if exists and exit without error.

# Read & Write Data

```
from pyspark.sql import SparkSession
from pyspark import SparkContext, SparkConf

spark = SparkSession.builder.appName('SparkReadWriteData').getOrCreate()

df1 = spark.read.format('csv').schema(var_schema).load('<input-file-path>')

df2 = df1.<some-transformation-1>

df3 = df2.<some-transformation-2>

df3.write.mode('append').format('parquet').save('<output-file-path>')
```

Let's write a small code that will read data from CSV file and write to PARQUET

# File Sources Configuration - CSV

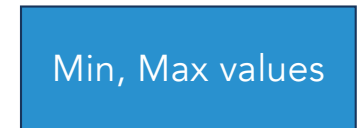
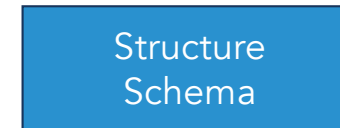
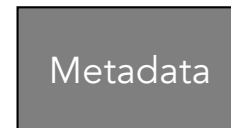
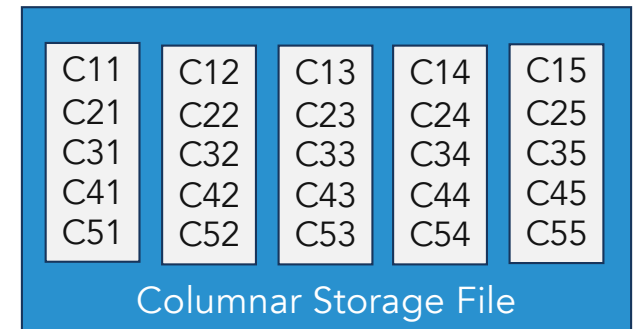
- CSV
  - sep – Separator for each field.
  - quote – Character used for string.
  - header – Whether CSV file contains header record or not. First line is used as column names.
  - inferSchema – Infer schema automatically from file data.
  - dateFormat – How date is stored in the input file. E.g. yyyy-MM-dd.
  - timestampFormat – Same as dateFormat.
  - mode – How to deal with corrupted records. PERMISSIVE, DROPMALFORMED, FAILFAST.
  - compression – bzip2, gzip, lz4, snappy, deflate while saving file.

# File Sources Configuration - JSON

- JSON
  - dateFormat – How date is stored in the input file. E.g. yyyy-MM-dd.
  - timestampFormat – Same as dateFormat.
  - mode – How to deal with corrupted records. PERMISSIVE, DROPMALFORMED, FAILFAST.
  - compression – bzip2, gzip, lz4, snappy, deflate while saving file.

# File Sources Configuration - Parquet

- PARQUET
  - mergeSchema – Sets whether we should merge schemas collected from all Parquet part-files.
  - compression – uncompressed, snappy, gzip, lzo, brotli, lz4, zstd.
- Configuration using SparkConf
  - spark.sql.parquet.outputTimestampType
  - spark.sql.parquet.filterPushdown – WHERE, Filter
  - spark.sql.parquet.aggregatePushdown
    - MIN, MAX (int, float, date, bool)
    - COUNT (all data types)





# File Sources Configuration - Avro

- PARQUET
  - avroSchema – Schema can be provided optionally.
  - compression.
  - Mode.
  - `spark.sql.avro.filterPushdown.enabled`

# Define Schema

- Define schema for CSV, JSON files.
- StructType, StructField in pyspark.sql.types library.
- Check schema after creating DataFrame from a file or otherwise.
- df.printSchema()

Let's use header, inferSchema options for Spark to create schema on its own. And then define our own schema.

```
from pyspark.sql.types import StructType, StructField

schema = StructType([
    StructField(<column-name>, <data-type>, <nullable-boolean>),
    StructField("cust_id", IntegerType(), True),
    StructField("cust_name", StringType(), True)
])
```

# Define Data Types

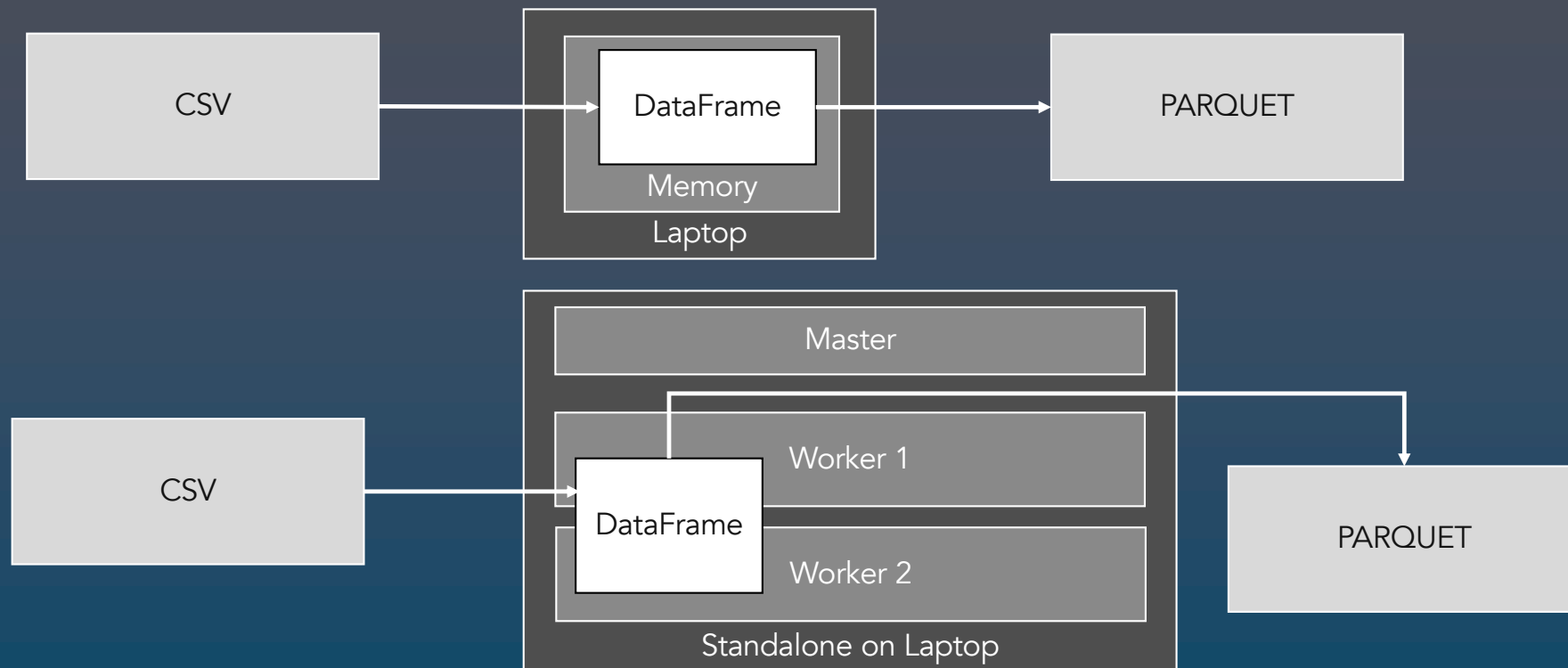
- Data types have to be imported from `pyspark.sql.types`

- `from pyspark.sql.types import ...`

- `DateType()`
    - `DecimalType()`
    - `FloatType()`
    - `IntegerType()`
    - `StringType()`
    - `VarcharType()`
    - `TimestampType()`
    - `CharType()`
    - `BinaryType()`
    - `IntegerType()`
    - `ArrayType()`
    - `DoubleType()`
    - `LongType()`

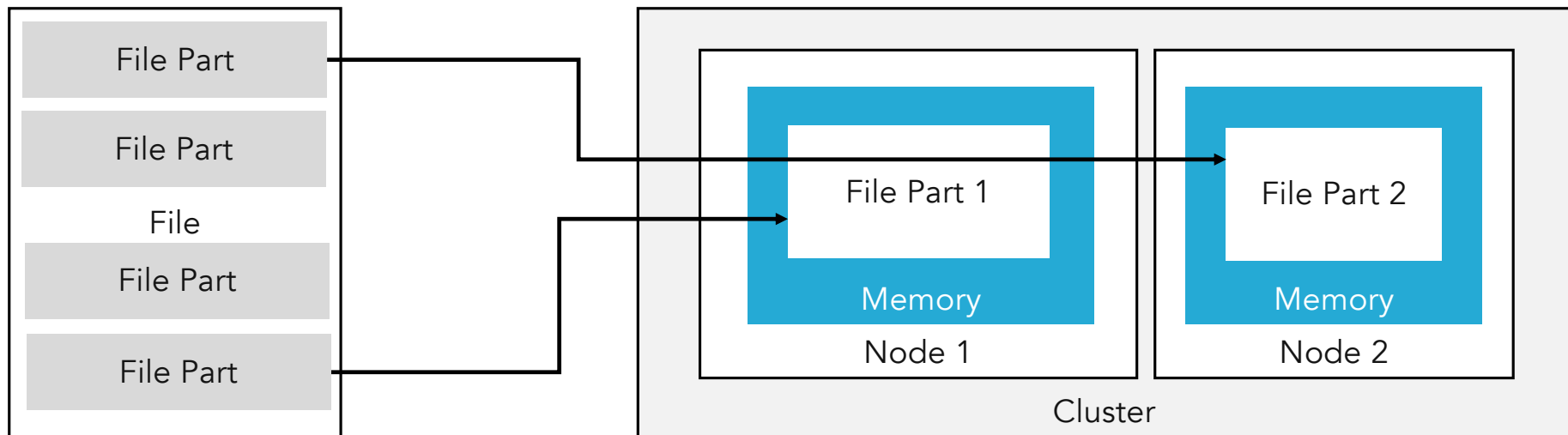
# HandsOn

- Read from a CSV file and write to a PARQUET file. Define the schema.



# How Such Big Files Fit in Memory

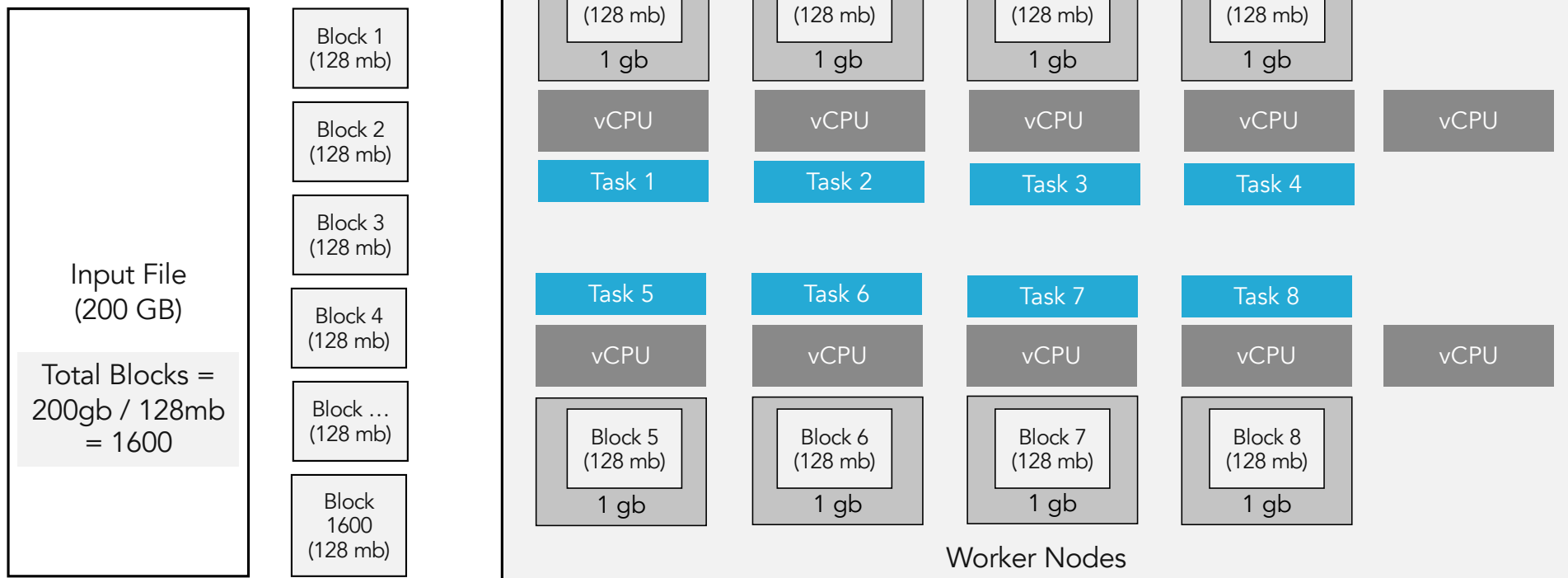
- File is read and load in memory.



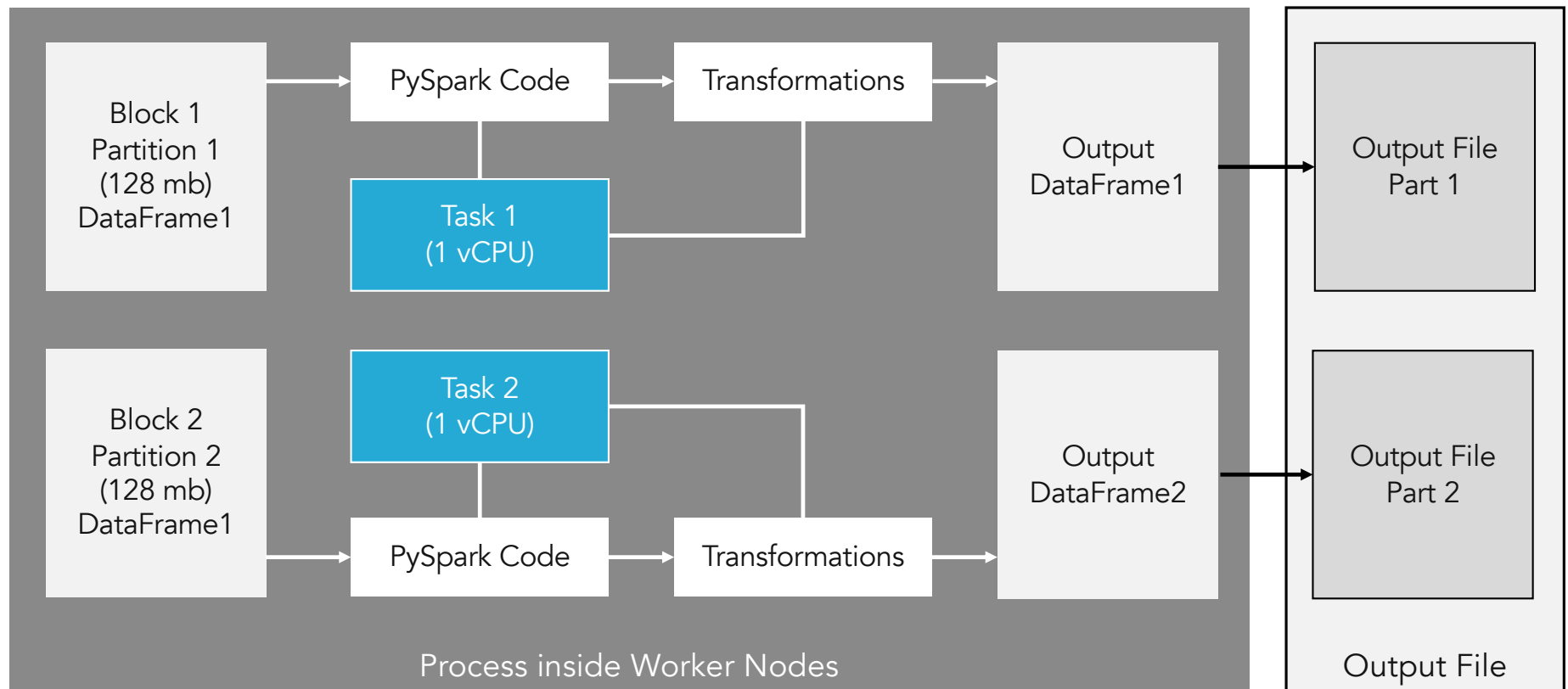
# Partitions, Tasks → Files, DataFrames

# Input Partitions

- HDFS Block Size – 128 MB

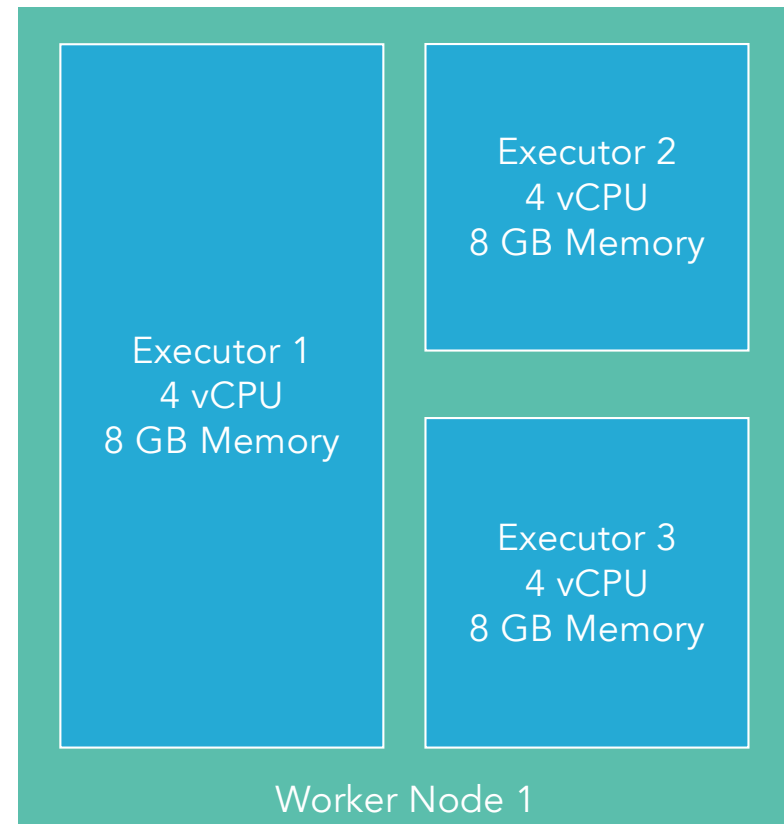
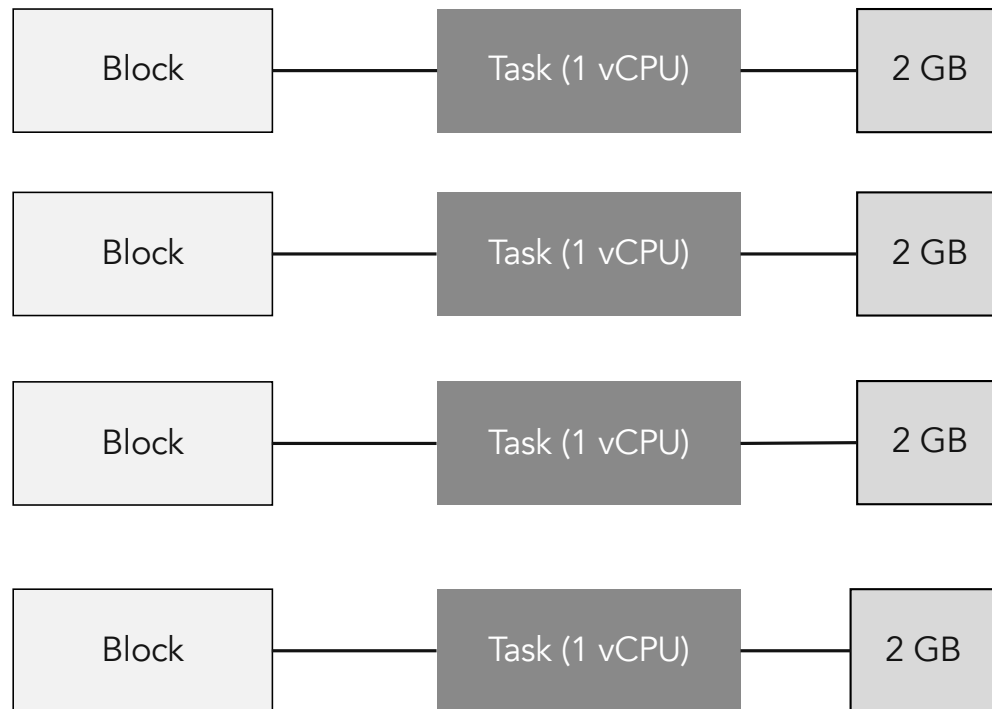


# Partitions, DataFrames, Tasks





# Partitions, Tasks, Executors



# Partitions, DataFrames, Tasks, Executors, Nodes

- Partition or Block or Split is the block size of input file system. HDFS, S3, GCS – 128 MB.
- Each partition is mapped to one Task.
- Task runs the PySpark user code. Code can contain one or more transformations.
- Each Task uses 1 vCPU.
- There can be multiple tasks per Executor depending on the no. of vCPUs per executor.
- Amount of memory per task depends on the total memory available per Executor.
- Executors reside in worker nodes.

# HandsOn

- Read CSV file and write to Parquet file, without any transformation.
- PyCharm.
- Execute using spark-submit on the standalone cluster in laptop.
- Explore using Spark UI on localhost.
- Execute using spark-submit on the standalone cluster in EC2 instance.
- Explore Spark UI on EC2 IPs.

# What if the file size is less

- Default Parallelism = no. of CPUs of the machine.
- `spark.sparkContext.defaultParallelism`
- `df.rdd.getNumPartitions()`
- Min Split Size = 4 MB.
- Mix of split size and no. of CPUs decide the no. of parallel tasks for smaller files.
- File Size = 50 MB. No. of CPUs = 10. Total max tasks =  $\min(50/4, 10) = 10$ .
- File Size = 20 MB. No. of CPUs = 10. Total max tasks =  $\min(20/4, 10) = 5$ .

# Splitable

- Snappy – Yes
- Bzip2 – Yes
- Gzip – No
- LZO – No
- CSV – Compressible and Splitable
- JSON – Compressible and Splitable

Will new DataFrames keep  
on occupying memory ??

# Summary

[www.3aayaam.in](http://www.3aayaam.in)

- Standalone Deployment Mode
- Spark UI
- DataFrames
- PySpark Module
- SparkSession
- File Sources
- Input Partitions, Tasks, Executors

A scenic landscape photograph featuring a vast field of tall, golden-green grass in the foreground. In the background, rolling hills and mountains are visible under a dramatic sky with orange, yellow, and blue clouds, suggesting a sunset or sunrise. The text "Thank You" and the website "www.3aayaam.in" are overlaid in white on the central part of the image.

Thank You

[www.3aayaam.in](http://www.3aayaam.in)